

End-to-End Lab: Terraform + Ansible on AWS

Real-Time Scenario: Building and Configuring a Secure 3-Tier Healthcare Web Application Platform

Prepared by: Phanindra Adinarayana Polisetty (Adi) **Level:** Intermediate-Advanced | **Cloud:** AWS | **Tools:** Terraform, Ansible
Estimated time: 3-4 hours

Lab Scenario

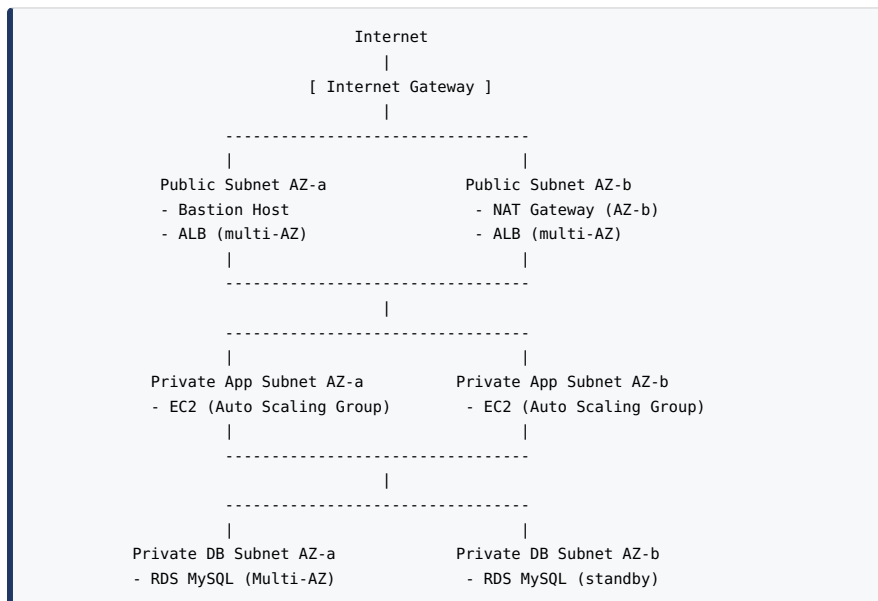
You are the DevSecOps engineer for a healthcare technology company. You've been asked to stand up a **secure, repeatable, 3-tier web application environment** on AWS that could later host a HIPAA-sensitive patient-facing application. The environment must be:

- Fully defined as code (no manual console clicks)
- Provisioned with **Terraform** (networking, compute, load balancer, database, IAM)
- Configured with **Ansible** (OS hardening, web server, application deployment, patching)
- Reachable only through a bastion host and a load balancer — no direct public access to app or database tiers

By the end of this lab you will have:

1. A custom VPC with public and private subnets across 2 Availability Zones
 2. A bastion host for secure SSH access
 3. An Auto Scaling Group of application servers in private subnets
 4. An Application Load Balancer distributing traffic to the app tier
 5. A MySQL RDS instance in an isolated private subnet
 6. Ansible dynamic inventory pulling live EC2 instances from AWS
 7. Ansible playbooks that harden, configure, and deploy an application to every instance — automatically, every time Terraform provisions new instances
-

Architecture Overview



Traffic flow: Internet → ALB (public subnets) → App servers (private subnets) → RDS (isolated private subnets). Admin access is via Bastion → SSH ProxyJump → App servers. No app or DB instance ever has a public IP.

Prerequisites

Requirement	Details
AWS account	With an IAM user/role that can create VPC, EC2, RDS, IAM, ELB resources
Terraform	v1.7+ installed locally (<code>terraform -version</code>)
Ansible	v2.15+ installed locally (<code>ansible --version</code>)
AWS CLI	Configured (<code>aws configure</code>) with access key/secret or SSO profile
SSH key pair	Created in AWS (or generate one in Scenario 1)
boto3	<code>pip install boto3 botocore</code> — required for Ansible's <code>aws_ec2</code> dynamic inventory plugin

Directory structure you'll build during this lab:

```
tf-ansible-lab/
├── terraform/
│   ├── backend.tf
│   ├── main.tf
│   ├── variables.tf
│   ├── vpc.tf
│   ├── security-groups.tf
│   ├── bastion.tf
│   ├── alb.tf
│   ├── asg-app.tf
│   ├── rds.tf
│   ├── iam.tf
│   └── outputs.tf
└── ansible/
    ├── ansible.cfg
    ├── inventory/
    │   └── aws_ec2.yml
    ├── group_vars/
    │   └── all.yml
    ├── site.yml
    └── roles/
        ├── hardening/tasks/main.yml
        ├── webserver/tasks/main.yml
        └── app_deploy/tasks/main.yml
```

Scenario 1 — Remote State Backend (S3 + DynamoDB Lock)

Why: In any real team environment, Terraform state must live remotely with locking — otherwise two engineers running `apply` at the same time will corrupt state. This is the first thing to set up, before any other resource.

Step 1.1 — Create the state bucket and lock table (one-time, run manually or via a bootstrap script):

```
aws s3api create-bucket --bucket adi-lab-tfstate-2026 \
  --region us-east-1

aws s3api put-bucket-versioning --bucket adi-lab-tfstate-2026 \
  --versioning-configuration Status=Enabled

aws dynamodb create-table \
  --table-name terraform-lock-table \
  --attribute-definitions AttributeName=LockID,AttributeType=S \
  --key-schema AttributeName=LockID,KeyType=HASH \
  --billing-mode PAY_PER_REQUEST
```

Step 1.2 — `terraform/backend.tf`:

```
terraform {
  required_version = ">= 1.7.0"

  backend "s3" {
    bucket      = "adi-lab-tfstate-2026"
    key         = "3tier-lab/terraform.tfstate"
    region      = "us-east-1"
    dynamodb_table = "terraform-lock-table"
    encrypt     = true
  }

  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}
```

Real-time gotcha: if two people `apply` at once without the DynamoDB lock table, you'll get silent state corruption that only surfaces days later as "resource already exists" errors. Always configure locking before writing any other resource.

Scenario 2 — VPC, Subnets, and Routing

Step 2.1 — terraform/variables.tf:

```
variable "aws_region"      { default = "us-east-1" }
variable "vpc_cidr"        { default = "10.20.0.0/16" }
variable "azs"             { default = ["us-east-1a", "us-east-1b"] }
variable "public_subnets" { default = ["10.20.0.0/24", "10.20.1.0/24"] }
variable "app_subnets"    { default = ["10.20.10.0/24", "10.20.11.0/24"] }
variable "db_subnets"     { default = ["10.20.20.0/24", "10.20.21.0/24"] }
variable "key_name"        { default = "adi-lab-key" }
variable "instance_type"   { default = "t3.micro" }
variable "db_username"     { default = "labadmin" }
variable "db_password"     { sensitive = true }
```

Step 2.2 — terraform/vpc.tf:

```
resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true
  tags = { Name = "lab-vpc" }
}

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id
  tags   = { Name = "lab-igw" }
}

resource "aws_subnet" "public" {
  count                = length(var.public_subnets)
  vpc_id               = aws_vpc.main.id
  cidr_block           = var.public_subnets[count.index]
  availability_zone     = var.azs[count.index]
  map_public_ip_on_launch = true
  tags = { Name = "lab-public-${count.index}" }
}

resource "aws_subnet" "app" {
  count                = length(var.app_subnets)
  vpc_id               = aws_vpc.main.id
  cidr_block           = var.app_subnets[count.index]
  availability_zone     = var.azs[count.index]
  tags = { Name = "lab-app-${count.index}" }
}

resource "aws_subnet" "db" {
  count                = length(var.db_subnets)
  vpc_id               = aws_vpc.main.id
  cidr_block           = var.db_subnets[count.index]
  availability_zone     = var.azs[count.index]
  tags = { Name = "lab-db-${count.index}" }
}

resource "aws_eip" "nat" {
  domain = "vpc"
}

resource "aws_nat_gateway" "nat" {
  allocation_id = aws_eip.nat.id
  subnet_id     = aws_subnet.public[0].id
  tags = { Name = "lab-nat" }
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
  tags = { Name = "lab-public-rt" }
}

resource "aws_route_table" "private" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block = "0.0.0.0/0"
  }
}
```

```
    nat_gateway_id = aws_nat_gateway.nat.id
  }
  tags = { Name = "lab-private-rt" }
}

resource "aws_route_table_association" "public" {
  count          = length(aws_subnet.public)
  subnet_id      = aws_subnet.public[count.index].id
  route_table_id = aws_route_table.public.id
}

resource "aws_route_table_association" "app" {
  count          = length(aws_subnet.app)
  subnet_id      = aws_subnet.app[count.index].id
  route_table_id = aws_route_table.private.id
}

resource "aws_route_table_association" "db" {
  count          = length(aws_subnet.db)
  subnet_id      = aws_subnet.db[count.index].id
  route_table_id = aws_route_table.private.id
}
```

Validate:

```
cd terraform
terraform init
terraform plan -out=vpc.plan
terraform apply vpc.plan
```

Real-time gotcha: a single NAT Gateway (as above) is fine for a lab, but in production put one NAT Gateway per AZ — otherwise an AZ failure takes out egress internet for every private subnet, not just its own.

Scenario 3 — Security Groups (Least Privilege, Tier by Tier)

terraform/security-groups.tf :

```
resource "aws_security_group" "bastion" {
  name     = "lab-bastion-sg"
  vpc_id   = aws_vpc.main.id

  ingress {
    description = "SSH from my IP only"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["<YOUR_PUBLIC_IP>/32"]
  }
  egress {
    from_port = 0; to_port = 0; protocol = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_security_group" "alb" {
  name     = "lab-alb-sg"
  vpc_id   = aws_vpc.main.id

  ingress {
    description = "HTTP from internet"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port = 0; to_port = 0; protocol = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_security_group" "app" {
  name     = "lab-app-sg"
  vpc_id   = aws_vpc.main.id

  ingress {
    description     = "HTTP from ALB only"
    from_port       = 80
    to_port         = 80
    protocol        = "tcp"
    security_groups = [aws_security_group.alb.id]
  }
  ingress {
    description     = "SSH from bastion only"
    from_port       = 22
    to_port         = 22
    protocol        = "tcp"
    security_groups = [aws_security_group.bastion.id]
  }
  egress {
    from_port = 0; to_port = 0; protocol = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_security_group" "db" {
  name     = "lab-db-sg"
  vpc_id   = aws_vpc.main.id

  ingress {
    description     = "MySQL from app tier only"
    from_port       = 3306
    to_port         = 3306
    protocol        = "tcp"
    security_groups = [aws_security_group.app.id]
  }
  egress {
    from_port = 0; to_port = 0; protocol = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

}

Real-time scenario check: this is the exact pattern auditors look for in a HIPAA/HITRUST review — every ingress rule points to a **security group**, not a CIDR block, except the bastion (locked to your IP) and ALB (intentionally public on 80). If you see `0.0.0.0/0` on an app or DB security group in a real environment, that's an immediate finding.

Scenario 4 — IAM Role for EC2 (Instance Profile, No Hardcoded Keys)

terraform/iam.tf :

```
resource "aws_iam_role" "app_role" {
  name = "lab-app-role"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action = "sts:AssumeRole"
      Effect = "Allow"
      Principal = { Service = "ec2.amazonaws.com" }
    }]
  })
}

resource "aws_iam_role_policy_attachment" "ssm" {
  role      = aws_iam_role.app_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore"
}

resource "aws_iam_instance_profile" "app_profile" {
  name = "lab-app-instance-profile"
  role = aws_iam_role.app_role.name
}
```

Why this matters: attaching the SSM policy means Ansible (and you) can reach instances via **SSM Session Manager** as a fallback even without the bastion — and it means no IAM access keys are ever stored on disk on the instance, satisfying the “no credential in code” control from your compliance checklist.

Scenario 5 — Bastion Host, ALB, and Auto Scaling Group of App Servers

terraform/bastion.tf :

```
resource "aws_instance" "bastion" {
  ami           = data.aws_ami.al2023.id
  instance_type = "t3.micro"
  subnet_id     = aws_subnet.public[0].id
  key_name      = var.key_name
  vpc_security_group_ids = [aws_security_group.bastion.id]
  tags = { Name = "lab-bastion" }
}

data "aws_ami" "al2023" {
  most_recent = true
  owners     = ["amazon"]
  filter {
    name   = "name"
    values = ["al2023-ami-*-x86_64"]
  }
}
```

terraform/alb.tf :

```
resource "aws_lb" "app" {
  name            = "lab-app-alb"
  internal        = false
  load_balancer_type = "application"
  security_groups = [aws_security_group.alb.id]
  subnets        = aws_subnet.public[*].id
}

resource "aws_lb_target_group" "app" {
  name     = "lab-app-tg"
  port     = 80
  protocol = "HTTP"
  vpc_id   = aws_vpc.main.id
  health_check {
    path            = "/health"
    interval        = 15
    healthy_threshold = 2
    unhealthy_threshold = 3
  }
}

resource "aws_lb_listener" "http" {
  load_balancer_arn = aws_lb.app.arn
  port              = 80
  protocol          = "HTTP"
  default_action {
    type = "forward"
    target_group_arn = aws_lb_target_group.app.arn
  }
}
```

terraform/asg-app.tf :

```
resource "aws_launch_template" "app" {
  name_prefix   = "lab-app-"
  image_id      = data.aws_ami.al2023.id
  instance_type = var.instance_type
  key_name      = var.key_name
  vpc_security_group_ids = [aws_security_group.app.id]

  iam_instance_profile {
    name = aws_iam_instance_profile.app_profile.name
  }

  tag_specifications {
    resource_type = "instance"
    tags = {
      Name = "lab-app-server"
      Role = "webapp"          # <-- Ansible dynamic inventory groups on this tag
    }
  }
}
```

```

    Env = "lab"
  }
}

resource "aws_autoscaling_group" "app" {
  name                = "lab-app-asg"
  desired_capacity    = 2
  min_size            = 2
  max_size            = 4
  vpc_zone_identifier = aws_subnet.app[*].id
  target_group_arns   = [aws_lb_target_group.app.arn]

  launch_template {
    id      = aws_launch_template.app.id
    version = "$Latest"
  }

  tag {
    key      = "Name"
    value    = "lab-app-server"
    propagate_at_launch = true
  }

  tag {
    key      = "Role"
    value    = "webapp"
    propagate_at_launch = true
  }
}

```

Real-time gotcha: the `Role = webapp` tag isn't decorative — it's the hook Ansible's dynamic inventory will use in Scenario 7 to automatically discover these instances without you ever typing an IP address into a file.

Scenario 6 — RDS MySQL in an Isolated Private Subnet

terraform/rds.tf :

```
resource "aws_db_subnet_group" "db" {
  name      = "lab-db-subnet-group"
  subnet_ids = aws_subnet.db[*].id
}

resource "aws_db_instance" "app_db" {
  identifier      = "lab-app-db"
  engine          = "mysql"
  engine_version  = "8.0"
  instance_class  = "db.t3.micro"
  allocated_storage = 20
  storage_encrypted = true
  db_subnet_group_name = aws_db_subnet_group.db.name
  vpc_security_group_ids = [aws_security_group.db.id]
  username        = var.db_username
  password         = var.db_password
  multi_az         = true
  skip_final_snapshot = true
  backup_retention_period = 7
  publicly_accessible = false
}
```

Step 6.1 — supply the DB password securely, never in a committed .tfvars file:

```
export TF_VAR_db_password=$(aws secretsmanager get-secret-value \
  --secret-id lab/db-password --query SecretString --output text)
```

Apply everything so far:

```
terraform plan -out=full.plan
terraform apply full.plan
```

Scenario 7 — Terraform Outputs Feeding Ansible

terraform/outputs.tf :

```
output "bastion_public_ip" {
  value = aws_instance.bastion.public_ip
}

output "alb_dns_name" {
  value = aws_lb.app.dns_name
}

output "db_endpoint" {
  value     = aws_db_instance.app_db.endpoint
  sensitive = true
}
```

Step 7.1 — Ansible needs boto3 and the AWS collection:

```
pip install boto3 botocore
ansible-galaxy collection install amazon.aws
```

Step 7.2 — ansible/ansible.cfg :

```
[defaults]
inventory      = ./inventory/aws_ec2.yml
host_key_checking = False
retry_files_enabled = False

[ssh_connection]
ssh_args = -o ProxyCommand="ssh -W %h:%p -q ec2-user@<BASTION_PUBLIC_IP>"
```

Step 7.3 — ansible/inventory/aws_ec2.yml — this is the dynamic inventory plugin that replaces a static list of IPs entirely:

```
plugin: amazon.aws.aws_ec2
regions:
  - us-east-1
filters:
  tag:Role: webapp
  instance-state-name: running
keyed_groups:
  - key: tags.Role
    prefix: role
hostnames:
  - private-ip-address
compose:
  ansible_host: private_ip_address
```

Step 7.4 — verify Ansible can see the live infrastructure Terraform just built:

```
cd ansible
ansible-inventory -i inventory/aws_ec2.yml --graph
ansible role_webapp -m ping
```

Real-time scenario: this is the step that most “Terraform + Ansible” tutorials skip and most interviewers ask about directly — how do you avoid maintaining a static `hosts` file that goes stale every time the Auto Scaling Group scales up/down or replaces an instance? The answer is the `aws_ec2` dynamic inventory plugin, filtering on the same tag Terraform applies at launch.

Scenario 8 — Ansible: OS Hardening, Web Server, and App Deployment

ansible/group_vars/all.yml :

```
app_port: 80
app_user: appuser
db_host: "{{ hostvars['localhost']['db_endpoint'] | default('') }}"
```

ansible/roles/hardening/tasks/main.yml — baseline CIS-style hardening, matching the “automated OS patching / CIS benchmark hardening” pattern from a real compliance programme:

```
- name: Ensure all packages are updated (patch management)
  ansible.builtin.dnf:
    name: "*"
    state: latest
    register: patch_result

- name: Disable root SSH login
  ansible.builtin.lineinfile:
    path: /etc/ssh/sshd_config
    regexp: '^PermitRootLogin'
    line: 'PermitRootLogin no'
    notify: restart sshd

- name: Enforce password-less sudo is disabled for app user
  ansible.builtin.lineinfile:
    path: /etc/sudoers.d/appuser
    create: yes
    line: "appuser ALL=(ALL) PASSWD: ALL"
    mode: '0440'

- name: Ensure firewalld is enabled and running
  ansible.builtin.systemd:
    name: firewalld
    enabled: true
    state: started

- name: Install and enable auditd for audit logging (HIPAA §164.312 audit control)
  ansible.builtin.dnf:
    name: audit
    state: present

- name: Enable auditd service
  ansible.builtin.systemd:
    name: auditd
    enabled: true
    state: started
```

ansible/roles/webserver/tasks/main.yml :

```
- name: Install nginx
  ansible.builtin.dnf:
    name: nginx
    state: present

- name: Deploy nginx reverse-proxy config
  ansible.builtin.template:
    src: nginx.conf.j2
    dest: /etc/nginx/conf.d/app.conf
    notify: restart nginx

- name: Deploy health check endpoint for ALB
  ansible.builtin.copy:
    content: "OK"
    dest: /usr/share/nginx/html/health
    mode: '0644'

- name: Ensure nginx is enabled and running
  ansible.builtin.systemd:
    name: nginx
    enabled: true
    state: started
```

ansible/roles/webserver/templates/nginx.conf.j2 :

```
server {
    listen 80;
    location /health {
        root /usr/share/nginx/html;
        try_files /health =200;
    }
    location / {
        proxy_pass http://127.0.0.1:{{ app_port }};
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

ansible/roles/app_deploy/tasks/main.yml :

```
- name: Create application user
  ansible.builtin.user:
    name: "{{ app_user }}"
    system: yes
    shell: /sbin/nologin

- name: Pull application artifact from S3
  ansible.builtin.command: >
    aws s3 cp s3://lab-app-artifacts/app-latest.tar.gz /opt/app.tar.gz
  args:
    creates: /opt/app.tar.gz

- name: Extract application
  ansible.builtin.unarchive:
    src: /opt/app.tar.gz
    dest: /opt/app
    remote_src: yes

- name: Template application environment file
  ansible.builtin.template:
    src: app.env.j2
    dest: /opt/app/.env
    owner: "{{ app_user }}"
    mode: '0600'

- name: Start application service
  ansible.builtin.systemd:
    name: app
    enabled: true
    state: restarted
```

ansible/site.yml — the master playbook that ties every role together:

```
- name: Configure and deploy the healthcare web app platform
  hosts: role_webapp
  become: yes
  roles:
    - hardening
    - webserver
    - app_deploy
```

Run it:

```
ansible-playbook site.yml
```

Scenario 9 — The End-to-End Loop (Terraform → Ansible, One Command)

In a real pipeline, you don't run Terraform and then remember to run Ansible manually. Wire them together with a `null_resource` and `local-exec` so any new instance is auto-configured the moment it's provisioned:

Add to `terraform/asg-app.tf` :

```
resource "null_resource" "run_ansible" {
  depends_on = [aws_autoscaling_group.app]

  provisioner "local-exec" {
    command = <<-EOT
    sleep 60
    cd ../ansible && ansible-playbook site.yml
    EOT
  }

  triggers = {
    asg_id = aws_autoscaling_group.app.id
  }
}
```

Run the full lab end-to-end:

```
cd terraform
terraform init
terraform apply -auto-approve
# Terraform provisions VPC -> IAM -> ALB -> ASG -> RDS
# then triggers Ansible automatically to harden + configure + deploy
```

Validate the whole platform:

```
curl http://$(terraform output -raw alb_dns_name)/health
# Expect: OK

curl http://$(terraform output -raw alb_dns_name)/
# Expect: your application's response, served through 2 hardened, load-balanced instances
```

Scenario 10 — Real-Time Troubleshooting Playbook

Symptom	Likely Cause	Fix
Error: state locked on terraform apply	Previous apply crashed, DynamoDB lock not released	<code>terraform force-unlock <LOCK_ID></code> after confirming no other apply is running
Ansible UNREACHABLE, timeout via bastion	ProxyCommand using stale bastion IP (bastion was replaced)	Re-fetch terraform output <code>bastion_public_ip</code> and update <code>ansible.cfg</code> — better: use SSM Session Manager as connection plugin instead of SSH ProxyCommand
ALB health checks failing, instances cycling	<code>/health</code> path not deployed before ASG attaches to target group	Ensure the <code>webserver</code> role's health endpoint task runs before instance is marked healthy; add ASG health check grace period (300s)
<code>ansible-inventory --graph</code> shows zero hosts	IAM role running Ansible lacks <code>ec2:DescribeInstances</code>	Attach <code>AmazonEC2ReadOnlyAccess</code> (or scoped equivalent) to the IAM identity running Ansible, not just the EC2 instance role
Terraform wants to recreate ASG instances on every apply	Launch template <code>user_data</code> or AMI data source resolving to a new value each run	Pin AMI with a specific <code>most_recent = false</code> + explicit filter, or use <code>ignore_changes</code> on non-critical attributes
RDS connection refused from app tier	Security group reference added after RDS already created	Security group rules referencing other SGs apply immediately — check <code>db.tf</code> was applied after <code>security-groups.tf</code> ; run <code>terraform plan</code> to confirm no drift
Config drift: manual console change reverted on next apply	Someone changed a resource outside Terraform	This is expected and correct behavior — track it as a compliance win: Terraform enforcing infrastructure back to its declared state is the same control pattern as OPA/Gatekeeper policy-as-code in Kubernetes

Scenario 11 — Cleanup (Avoid Ongoing AWS Charges)

```
cd terraform
terraform destroy -auto-approve
```

Then remove the one-time bootstrap resources if you no longer need them:

```
aws dynamodb delete-table --table-name terraform-lock-table
aws s3 rb s3://adi-lab-tfstate-2026 --force
```

Real-time reminder: always run `terraform plan` before `destroy` in a shared account to confirm you're not about to tear down another team's resources that happen to share the state file.

What This Lab Demonstrates (Talking Points for Interviews)

- **Terraform:** modular resource design, remote state with locking, secrets kept out of version control, IAM least privilege via instance profiles, security groups scoped tier-to-tier instead of open CIDR ranges.
- **Ansible:** dynamic inventory against live AWS tags (no static host files), role-based playbook structure, OS hardening mapped to a real compliance control (HIPAA §164.312 audit logging via auditd), idempotent configuration management.
- **Integration:** a single `terraform apply` triggers full configuration management — the same “infrastructure to running application” loop used in real CI/CD pipelines, just without the pipeline wrapper.
- **Operational maturity:** a troubleshooting table addressing the failure modes that actually occur in production — state locks, ASG health-check races, IAM permission gaps, and drift — rather than only the “happy path.”